
RAPPORT DE PROJET : **DÉVELOPPEMENT D'UNE APPLICATION** **DE GESTION DE BLOG**

Unité d'Enseignement : INF222 - Programmation Web

Noms : BILOA NGUEMA ALPHONSE KHAREL

Matricule : 24F2549

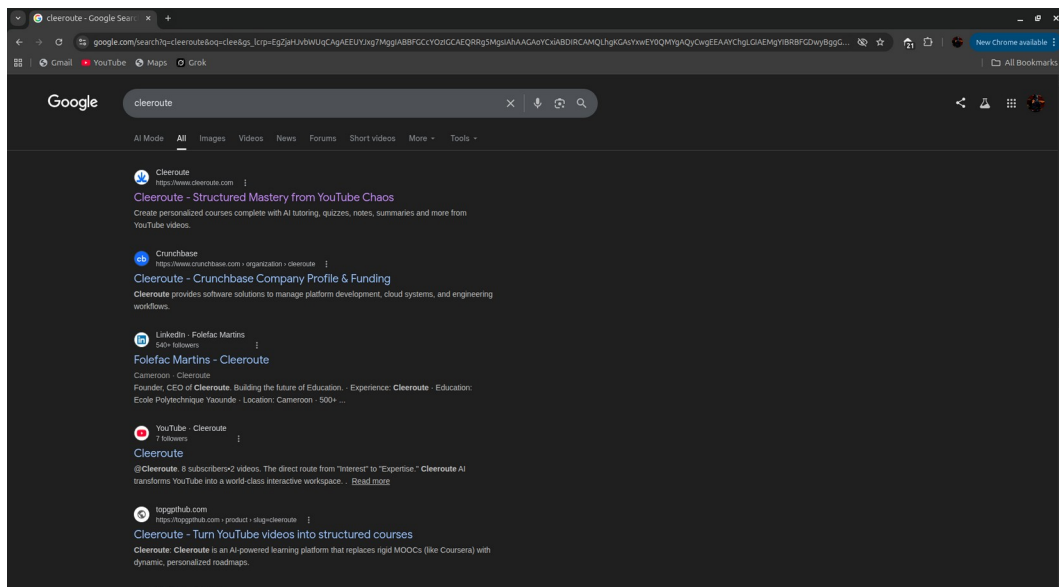
Lien Git : <https://github.com/Khare1154/My-Blog-API>

INTRODUCTION

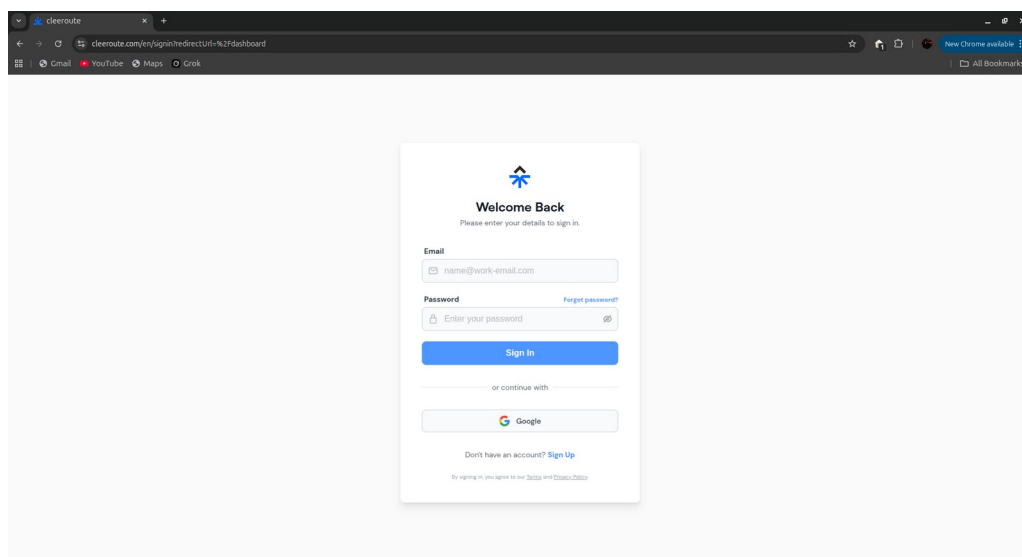
Ce rapport presente l'utilisation de la plateforme CleeRoute, la conception d'une API Backend d'une app de gestion de blog, et les critique sur la plateforme cleeroute.

PARTIE 1: Utilisation de CleeRoute

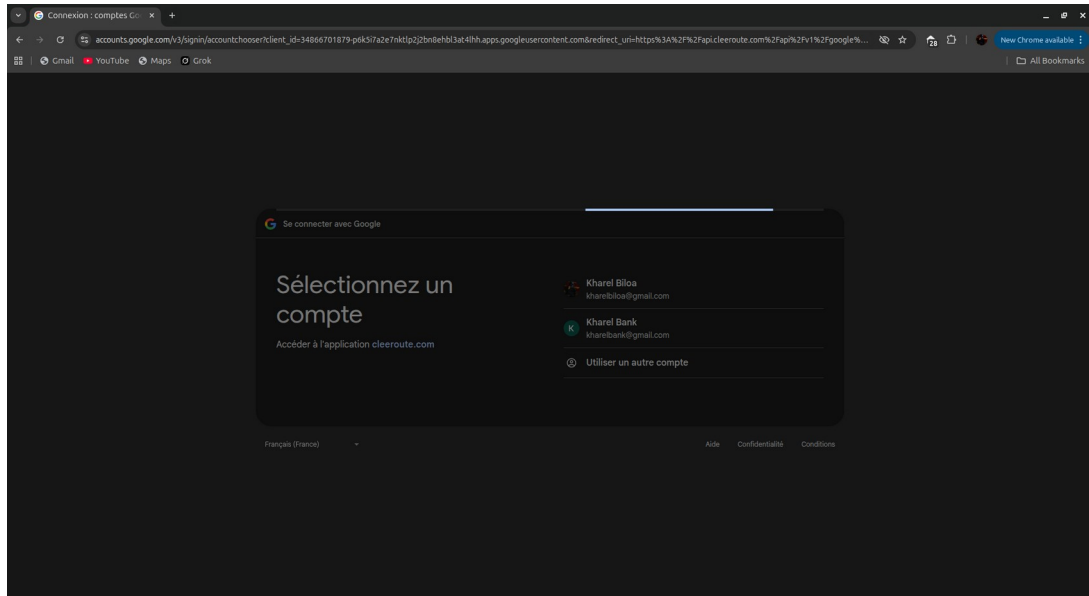
Ici, je recherche la plateforme CleeRoute sur mon navigateur.



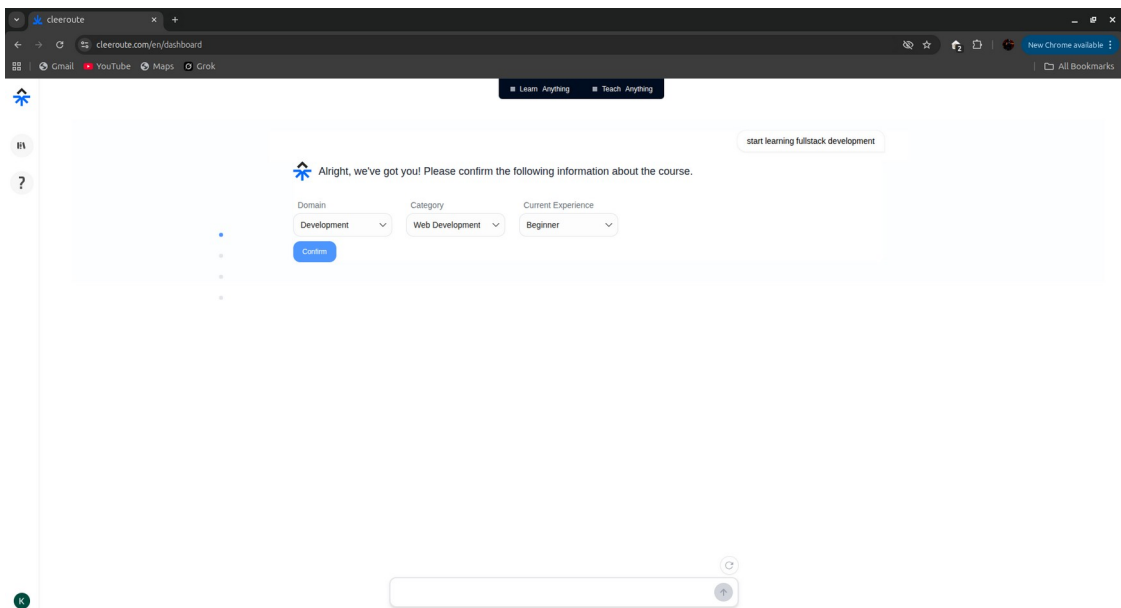
Ici, j'arrive sur l'écran de connexion de CleeRoute.



Ici, je me connecte directement avec Google.



Ici, je suis sur l'écran principal de CleerRoute et je saisis le domaine d'étude.



Ici, je précise a CleeRoute un projet qui accompagne mon apprentissage

The screenshot shows the CleeRoute dashboard at the URL `cleeroute.com/en/dashboard`. The page has a dark header with navigation links for "Learn Anything" and "Teach Anything". On the left, there is a sidebar with a blue star icon, a "BI" button, a question mark icon, and a green "K" button. The main content area is white and contains a form titled "Alright, we've got you! Please confirm the following information about the course." The form includes three dropdown menus: "Domain" (set to "Development"), "Category" (set to "Web Development"), and "Current Experience" (set to "Beginner"). Below these, there is a text input field with the placeholder "To help me tailor your Fullstack Development Fundamentals course, what is your ultimate learning goal, and what specific project or application do you envision building with your new skills?". A text box contains the answer "my specific project is developing an API blog backend app". At the bottom of the form, there is a text input field with the placeholder "Type your answer here..." and a submit button with an upward arrow.

CleeRoute me génère un parcours d'apprentissage adapté à mon projet

The screenshot shows the CleeRoute dashboard at the URL `cleeroute.com/en/dashboard`. The page has a dark header with navigation links for "Learn Anything" and "Teach Anything". On the left, there is a sidebar with a blue star icon, a "BI" button, a question mark icon, and a green "K" button. The main content area is white and contains a section titled "Pick a Path". Below this, there are five cards: "Backend Development", "MERN Stack Development", "Fullstack Development", "React Of Both worlds", and "Fullstack Development". The "Backend Development" card is selected and expanded, showing a list of topics: "01 JavaScript Essentials for Backend", "02 Node.js and NPM Fundamentals", "03 Express.js: Routing and Middleware", and "04 Handling User Input and Sessions". The "01 JavaScript Essentials for Backend" topic is highlighted, and a video thumbnail for "Master JavaScript Essentials for Backend Development | Part 1" is shown. At the bottom of the expanded card, there is a blue "Start Learning" button.

Ici, je suis les cours proposés par CleeRoute.

The screenshot displays the CleeRoute website interface. On the left, a video player shows a JavaScript console with the output `> obj { 'name': 'harsh' }`. Below the video, there are transcripts in Hindi. On the right, a sidebar lists the course sections: **JavaScript Essentials for Backend**, **Node.js and NPM Fundamentals**, **Express.js: Routing and Middleware**, **Handling User Input and Sessions**, **MongoDB and Mongoose for Database Manag...**, **CRUD Operations with MongoDB and EJS**, **Authentication and Authorization Strategies**, **Data Association in MongoDB**, **Handling File Uploads with Multer**, **Full-Stack Project: Scatch - Premium Bag Shop**, and **Advanced Backend Project: Chess.com Clone**. The top navigation bar includes links for **Course Content**, **Quizzes**, **Notes**, and **Sources**, along with **AI Chat** and **Upgrade** buttons.

Master Backend Development Series: NodeJS, MongoDB, Expre...

SHARE

Lite Version

UPGRADE

Course Content

Quizzes

Notes

Sources

AI Chat

QUIT

Quiz in progress...

1/5 You're building a REST API with Express.js and need to handle incoming JSON data in a POST request. Which middleware is essential for parsing this JSON body?

Skip

A express.static()

B express.json()

C express.urlencoded()

D helmet()

Hint



Master Backend Development Series: NodeJS, MongoDB, Expre...

SHARE

Lite Version

UPGRADE

Course Content

Quizzes

Notes

Sources

AI Chat

My Notes

Cancel

B I

To build a rest API with

Notes are saved automatically to your profile.

Chat

Test Knowledge

Create Summary

Take Notes

Ask anything

Lite

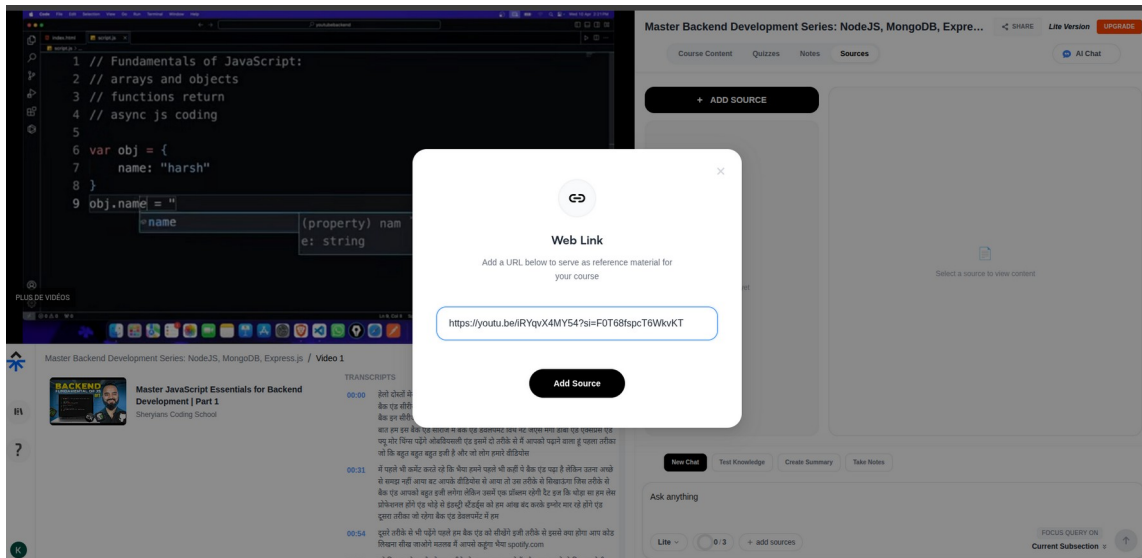
0 / 3

+ add sources

FOCUS QUERY ON

Current Subsection

Ici j'ajoute des ressources venant de YouTube pour approfondir mes connaissances du sujet



PARTIE 2: Backend de l'Application de Blog

STACK TECHNIQUE UTILISÉE

Pour cette partie, les technologies suivantes ont été implémentées :

- **Langage** : Python 3.10+.
- **Framework** : **FastAPI**, choisi pour sa rapidité d'exécution et sa gestion native de l'asynchronisme.
- **ORM (Object-Relational Mapping)** : **SQLAlchemy**, utilisé pour interagir avec la base de données sans écrire de requêtes SQL brutes.
- **Base de données** : SQLite (fichier `blog.db`), une base de données légère et auto-contenue, idéale pour ce projet.
- **Validation** : Pydantic, pour définir les schémas de données (Entrées/Sorties).

STRUCTURE ET FONCTIONNEMENT DE L'API

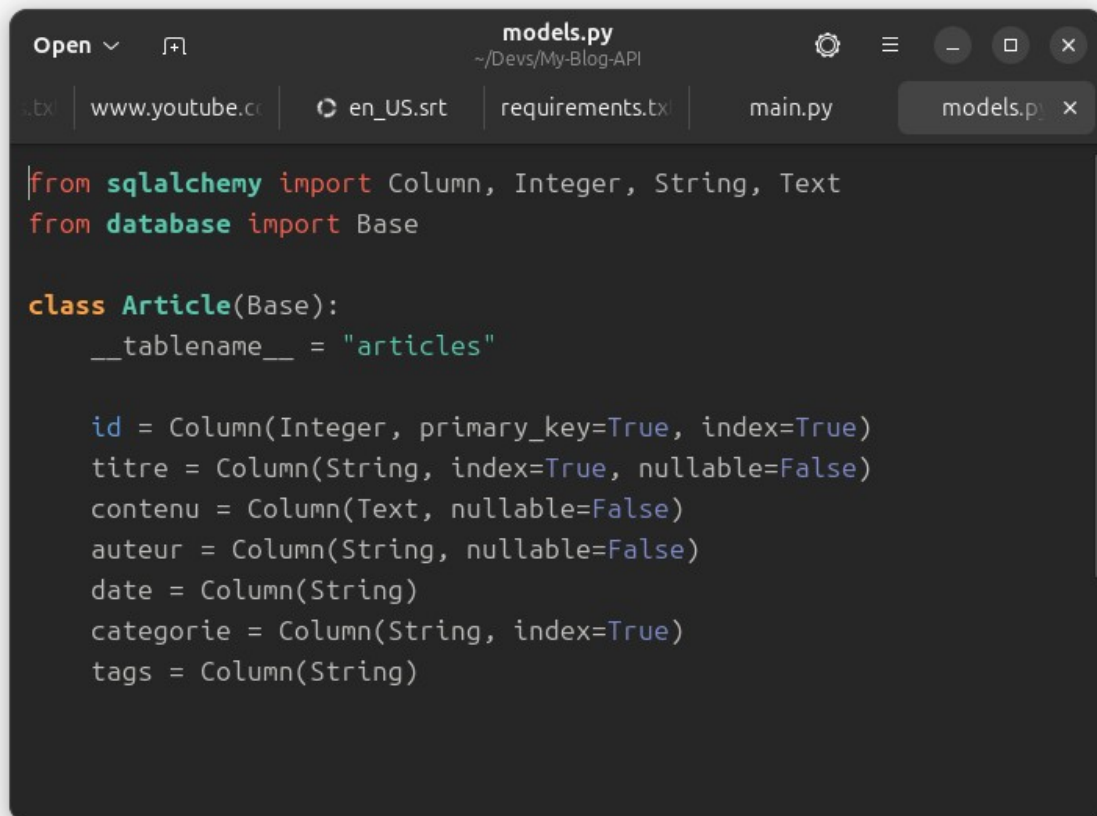
Le Backend expose plusieurs points de terminaison (endpoints) permettant de réaliser les opérations CRUD :

- **POST /api/articles** : Reçoit les données JSON d'un nouvel article et les enregistre en base de données.

- **GET /api/articles** : Récupère la liste complète de tous les articles stockés.
- **GET /api/articles/{id}** : Récupère les détails d'un article spécifique via son identifiant unique.
- **PUT /api/articles/{id}** : Met à jour les informations d'un article existant.
- **DELETE /api/articles/{id}** : Supprime définitivement un article de la base de données.
- **GET /api/articles/search** : Permet de filtrer les articles par mots-clés.

GESTION DE LA PERSISTANCE (BASE DE DONNÉES)

Le fichier `models.py` définit la structure de la table `articles` (Titre, Auteur, Contenu, Catégorie, Date, Tags). Le fichier `crud.py` contient la logique Python pour manipuler ces données (ajouter, modifier, supprimer)



```
models.py
~/Devs/My-Blog-API

www.youtube.co | en_US.srt | requirements.txt | main.py | models.py x

from sqlalchemy import Column, Integer, String, Text
from database import Base

class Article(Base):
    __tablename__ = "articles"

    id = Column(Integer, primary_key=True, index=True)
    titre = Column(String, index=True, nullable=False)
    contenu = Column(Text, nullable=False)
    auteur = Column(String, nullable=False)
    date = Column(String)
    categorie = Column(String, index=True)
    tags = Column(String)
```



```
Open ▾ | crud.py
~/DevOps/ReqAPI

from sqlalchemy.orm import Session
import models, schemas

def get_articles(db: Session, categorie: str = None, date: str = None):
    query = db.query(models.Article)
    if categorie:
        query = query.filter(models.Article.categorie == categorie)
    if date:
        query = query.filter(models.Article.date == date)
    return query.all()

def get_article(db: Session, article_id: int):
    return db.query(models.Article).filter(models.Article.id == article_id).first()

def search_articles(db: Session, query_text: str):
    # Si le texte est un chiffre, on cherche l'ID exact
    if query_text.isdigit():
        return db.query(models.Article).filter(models.Article.id == int(query_text)).all()

    # Sinon, on cherche dans le titre ou le contenu
    return db.query(models.Article).filter(
        (models.Article.titre.contains(query_text)) |
        (models.Article.contenu.contains(query_text))
    ).all()

def create_article(db: Session, article: schemas.ArticleCreate):
    db_article = models.Article(**article.model_dump())
    db.add(db_article)
    db.commit()
    db.refresh(db_article)
    return db_article

def update_article(db: Session, article_id: int, article_data: schemas.ArticleUpdate):
    db_article = get_article(db, article_id)
    if db_article:
        update_data = article_data.model_dump(exclude_unset=True)
        for key, value in update_data.items():
```

Une partie du fichier crud.py.

PARTIE 3: Analyse Critique

Globalement, je trouve que c'est une très bonne application qui fonctionne parfaitement. Elle sera réellement utile pour les étudiants souhaitant approfondir leurs connaissances sur des sujets précis. Cependant, j'ai relevé quelques points qui pourraient être améliorés :

- **Pertinence pédagogique des vidéos :** Dans certains cas, les vidéos proposées ne correspondent pas tout à fait aux besoins d'un débutant. Par exemple, lors d'une session où j'ai choisi d'apprendre le **DevOps** (sans sélectionner de projet spécifique), les vidéos suggérées étaient d'un niveau un peu trop avancé, alors que j'avais pourtant sélectionné le niveau "**Beginner**" (Débutant).
- **Structure de l'apprentissage :** Je pense qu'il serait préférable de commencer par une explication théorique complète du concept de DevOps avant de passer à la pratique. L'absence de cette introduction peut perdre un débutant complet, comme un élève sortant tout juste du secondaire